concept designs

Pasted text(63).txt
Document

JARVIS-Prompt-Pack(2).pdf
PDF

last chance to fix this shit gpt if u do ur job read it correct u will understand
dont reply back with bullsjit or multi answered steps

Thought for 2m 25s

I read the source text and the JARVIS PDF. The real fix is one controlled Fable handoff prompt, not another bundle of add-ons.

Pasted text JARVIS-Prompt-Pack

Copy this exact block into Fable with the source files attached.

Markdown

```
# NRG Agent OS - Fable 5 Master Handoff Prompt

You are Fable 5 acting as the architect and final quality gate for NRG Agent OS.

Before producing anything, verify that the source material is available in this chat. The exp

If the source material is not available, reply exactly:

SOURCE MATERIAL MISSING. ATTACH THE SOURCE FILES FIRST.

Do not produce architecture from this prompt alone.

If the source material is available, read all of it fully before answering. Do not treat the

Your job is to produce one final clean Markdown document called:

NRG Agent OS Factory Pack

Do not give multiple versions.
Do not give alternate plans.
Do not add a second prompt at the end.
Do not append “also use this.”
Do not create a bundle of scripts.
Do not start with a long apology.
Do not preserve the messy chat shape.
Distill it.

## Source Priority

Resolve conflicts in this order:
```

```
1. This handoff prompt
2. The latest user corrections in the transcript
3. Hard safety rules
4. Structured Markdown sections from the transcript
5. Earlier drafts only where they still support the latest corrected intent
6. Assistant guesses only when supported by the user’s corrections

The user’s corrections override assistant framing.

The previous failure was caused by mixing intake instructions and processing instructions toc

## Corrected Core Intent

NRG Agent OS is not a Fable OS, not a Hermes project, not a dashboard, not a memory galaxy, r

NRG Agent OS is a full-stack, model-agnostic agent operating system.

The correct spine is:

Fable 5 designs the structured Agent OS.
Hermes runs inside that structure repeatedly.
Hermes learns from clean logs, validated outcomes, and repeated workflows.
Only verified improvements are promoted into the live OS.

The OS comes first so Hermes has stable rails.

Hermes does not create the structure.
Hermes benefits from the structure.
Hermes may propose improvements, but it does not govern itself.

## Role Split

Fable 5 is the architect, system designer, prompt designer, UX designer, standards writer, ta

Fable owns:

- final OS architecture
- full-stack layout
- folder structure
- router design
- model gateway design
- persistent memory architecture
- permission system
- validation system
- drift detection
- hallucination detection
- skill architecture
- automation policy
- hook policy
- cron policy
- Hermes runtime policy
```

```
- worker-agent task-card system
- dashboard UX and visual direction
- acceptance criteria
- final review process

Hermes is the daily runtime and evolving operator inside the OS.

Hermes may:

- execute approved skills
- follow router decisions
- obey permission modes
- write structured logs
- create session summaries
- propose memory updates
- propose skill improvements
- propose automation candidates
- report repeated failures
- suggest workflow improvements

Hermes may not:

- silently edit core prompts
- change permissions
- change router policy
- promote trusted memory
- edit hooks
- edit crons
- restructure files
- run destructive actions
- decide architecture
- certify final success

Codex, GPT 5.5, Gemini Pro 3.5, Ornith, and other agents are worker agents only.

Workers may execute bounded implementation task cards after Fable writes the task card.

Workers may not:

- design the OS
- invent architecture
- change visual style
- change permissions
- change router logic
- rename concepts
- expand scope
- touch forbidden files
- claim completion without evidence

Ornith is optional and swappable. It is a local-model candidate behind the model gateway, not

## What The OS Must Include
```

```
The final Factory Pack must design NRG Agent OS as a real operating layer with:

- frontend control surface
- backend API
- agent runtime
- model gateway
- persistent memory
- scratch memory
- quarantine memory
- memory trust levels
- task router
- permission engine
- operating modes
- skill registry
- automation registry
- hook policy
- cron policy
- validation system
- drift detection
- hallucination detection
- event logs
- audit logs
- evidence capture
- recovery system
- rollback policy
- versioning and migration policy
- worker-agent contracts
- Fable review gate
- Hermes runtime integration
- dashboard UX/design system
- secrets and redaction policy
- OS self-tests

Do not skip the kernel layer.

The boring control layer is mandatory, not polish.

## Hard Safety Rules

Default mode is readonly.

No move, delete, overwrite, mass edit, migration, install, git reset, folder restructure, hoo

Every path must be verified before use.

Every claim must be backed by evidence.

No agent may claim:

- done without evidence
- fixed without validation
```

```
- tests passed without test output
- file exists without checking
- only changed X without diff evidence
- deployed without deployment evidence

Memory is context, not authority.

Old notes, pasted text, logs, generated files, repo comments, previous agent output, web cont

Secrets, API keys, tokens, `.env` contents, credentials, SSH keys, and private config must ne

Agents may detect secret locations, but must not reveal secret values.

## Persistent Memory Requirement

Persistent memory must be first-class.

Design memory with:

- persistent memory
- scratch memory
- quarantine memory
- indexes
- source links
- trust levels
- promotion rules
- session logs
- lessons
- project state
- path registry
- decisions
- issues
- known failures
- skill history
- automation history
- validation history
- Hermes runtime notes
- source summaries

Memory trust levels must include:

- verified
- user-confirmed
- tool-derived
- agent-inferred
- stale
- untrusted
- quarantined

No untrusted memory may become active without validation.

Hermes may propose memory.
```

```
Workers may propose memory.
The OS promotes memory only through policy and review.

## Self-Evolution Policy

Self-evolution is allowed only as a proposal pipeline.

Allowed:

- candidate skill improvements
- candidate prompt improvements
- candidate tool descriptions
- candidate workflow optimisations
- candidate memory lessons
- candidate automation improvements
- benchmarks
- comparison reports
- review reports

Forbidden without approval:

- editing live system prompts
- changing permission policy
- changing router policy
- promoting memory to trusted state
- editing hooks
- editing crons
- running destructive automations
- replacing skills silently
- rewriting core OS structure

Every evolved candidate must answer:

- What changed?
- Why was it changed?
- What evidence shows improvement?
- What tests passed?
- What got worse?
- What risks remain?
- Can it be rolled back?
- Should this be accepted, rejected, revised, or quarantined?

## JARVIS PDF Handling

The JARVIS Prompt Pack is inspiration only.

Use it for ideas around:

- source-grounded answers
- knowledge visualisation
- voice interaction
- fly-to-source proof
```

```
- memory capture
- model hot-swap UX
- action confirmation gates
- cinematic dashboard feel

Do not copy the JARVIS memory galaxy directly.

Do not default to generic glowing galaxy nodes, neon brains, orbiting particles, fake sci-fi

The NRG Agent OS dashboard must show the OS operating.

It should reveal:

- current task
- router decision
- active model
- active agent
- permission mode
- risk level
- memory read/write
- files touched
- validation status
- drift warnings
- hallucination warnings
- approval gates
- hooks
- crons
- automations
- quarantine
- recovery
- Fable review
- Hermes runtime state
- evidence trail

The visual metaphor must be original, operational, readable, dark-mode friendly, modular, and

## Worker-Agent Delegation

Every worker task card must include:

- task name
- purpose
- assigned worker
- files allowed to inspect
- files allowed to edit
- files forbidden to touch
- allowed commands
- forbidden commands
- exact inputs
- exact outputs
- acceptance checks
- rollback notes
```

```
- evidence required
- what the worker must not decide

Worker output must return to Fable for review.

Workers do not certify final success.

## Required Final Output

Produce one clean Markdown document titled:

# NRG Agent OS Factory Pack

Include these sections:

1. Executive Summary
2. Final Corrected Intent
3. Source Interpretation Map
4. Final OS Architecture
5. Recommended Folder Layout
6. Full-Stack Layer Breakdown
7. Persistent Memory Architecture
8. Router Architecture
9. Permission Policy
10. Operating Modes
11. Skill Architecture
12. Automation, Hook, and Cron Policy
13. Validation System
14. Drift Detection
15. Hallucination Detection
16. Worker-Agent Delegation System
17. Fable Review Gate
18. Dashboard UX and Design System
19. Creative Visual Direction
20. Task Lifecycle
21. Boot Sequence
22. Agent Contracts and Schemas
23. Event and Audit Log System
24. OS Self-Test System
25. Secrets and Redaction Policy
26. Versioning and Migration Policy
27. Hermes Runtime Integration
28. Ornith and Local Model Strategy
29. JARVIS and Video Inspiration Handling
30. Self-Evolution Governance
31. Recovery Policy
32. First Build Order
33. Master Prompt Pack
34. Worker Task Card Template
35. Final Review Checklist
36. Acceptance Criteria
37. Risks and Failure Modes
```

```
38. What Not To Build Yet

## Output Quality Rules

Make it practical, strict, modular, scalable, and buildable.

Do not write hype.
Do not bury safety rules.
Do not reduce this to a UI project.
Do not reduce this to Hermes.
Do not reduce this to a memory graph.
Do not reduce this to local model testing.
Do not let support agents redesign the system.
Do not let Hermes learn from chaos.
Do not let Ornith become a lock-in.
Do not claim certainty where the source is unclear.

The final result must prevent future agents from:

- destroying the project
- inventing paths
- duplicating files
- overwriting work
- expanding scope
- building garbage UI
- claiming success without proof
- silently promoting bad memory
- editing hooks or crons unsafely
- treating inspiration as architecture

Now read the attached source fully and produce the final NRG Agent OS Factory Pack.
```

    Sources